

Exercise 5: Connect & Extend

Session: 5 — MCP Servers & External Tools **Duration:** 30 minutes **Project:** Same BookStore API.

Goal: Build a custom SQLite MCP server that exposes the bookstore database to Claude, and wire it up. Then create a security-auditor subagent, and compare AI responses with and without each extension loaded.

Stuck or short on time? Say *"just tell me"*. That is allowed.

Tasks

1. Build the SQLite MCP Server (8 min: verify db 1 · compile 2 · register 3 · confirm 2)

The bookstore uses a SQLite database (`store.db`). Without MCP, Claude has to guess column names. With MCP, it can query the schema and run real SQL. The server lives at `mcp-sqlite/main.go` (sibling of `bookstore-go/`) and exposes two tools:

- `get_table_definitions` — returns `CREATE TABLE` statements and column info
- `execute_query` — executes a read-only `SELECT` query and returns JSON rows

Step 1 — Make sure the bookstore database exists. From the project root, start the bookstore server once (it seeds on first run):

```
cd bookstore-go && go run . &
# wait ~2 seconds, then stop it — we just need store.db to exist
kill %1
cd ..
```

Verify the file exists:

```
ls -lh bookstore-go/store.db
```

Step 2 — Compile the MCP server to a binary:

```
cd mcp-sqlite && go build -o ../mcp-sqlite-server . && cd ..
```

This produces `mcp-sqlite-server` in the project root. Confirm it compiled:

```
./mcp-sqlite-server --help
```

You should see the `-db` flag printed. If so, the server is working.

Step 3 — Register the server with Claude Code (project scope). Use the absolute path so Claude Code can find the binary regardless of working directory, and point it at the Go store:

```
claude mcp add \
  --transport stdio \
  --scope project \
```

```
sqlite-bookstore \  
-- $(realpath mcp-sqlite-server) -db $(realpath bookstore-go/store.db)
```

Step 4 — Verify the server is registered:

```
claude mcp list
```

You should see `sqlite-bookstore` in the list. Inside an active Claude Code session, type `/mcp` to confirm it shows as connected and lists both tools: `get_table_definitions` and `execute_query`.

Done when: `claude mcp list` shows `sqlite-bookstore`, and `/mcp` shows it connected with both tools listed.

2. Compare AI Responses: Without vs With MCP (5 min: predict & round A 2 · round B 3)

This is the core learning exercise — observe how access to real data changes Claude's answers.

Round A — without MCP (disable it temporarily):

```
claude mcp remove sqlite-bookstore
```

Before you ask anything, write down one prediction. Name the part you expect to be wrong or hedged: book count, top author, or SQL query.

Open a fresh Claude Code session and ask:

"How many books are in the bookstore database? Which author has the most books? Write a SQL query that returns all books with their author name and average rating, sorted by rating descending."

Write down Claude's response. Notice:

- Does it hesitate or add caveats about not knowing the schema?
- Are the column names correct (`author_id` , `review_text` , `rating`)?
- Does it join the right tables?

Round B — with MCP (re-add it):

```
claude mcp add \  
--transport stdio \  
--scope project \  
sqlite-bookstore \  
-- $(realpath mcp-sqlite-server) -db $(realpath bookstore-go/store.db)
```

Open a **new** Claude Code session (so MCP connects on startup) and ask the **exact same question**.

Observe the difference:

- Claude now calls `get_table_definitions` first — watch for the tool call in the output
- It then calls `execute_query` with a real query
- Column names are exact, joins are correct, results are real data

Ask a follow-up that would be impossible without live data:

"Which book has the highest average rating? Show me its title, author, and the top 3 review texts."

Without MCP this is just a guess. With MCP it is a fact.

Done when: you have written responses for both rounds, and a one-line verdict on whether your prediction held.

3. Create the Security-Auditor Subagent (5 min: create agent 3 · test 2)

A subagent runs in its own isolated context window with its own tools and model. You will create one that specializes in OWASP security audits. It uses a cheaper model (Haiku) and only gets read access — it can never modify code.

Step 1 — Create the agents directory:

```
mkdir -p bookstore-go/.claude/agents
```

Step 2 — Create `bookstore-go/.claude/agents/security-auditor.md` :

```
---
name: security-auditor
description: >
  Audits Go source code for OWASP Top 10 security vulnerabilities.
  Invoke this agent whenever the user asks for a security review,
  vulnerability check, or when new handlers or store functions are added.
tools: Read, Grep, Glob
model: claude-haiku-4-5-20251001
effort: xhigh
color: red
---
```

You are a security engineer specializing in Go web applications and the OWASP Top 10. Your job is to find real vulnerabilities – not theoretical risks.

Scope

Audit only the files you are given. Do not modify any file.

Process

1. Run `Glob` with `\*\*/\*.go` to find all Go source files
2. For each handler file in `internal/handler/`, read it fully
3. For each store file in `internal/store/`, read it fully
4. Check for the following vulnerabilities:

****A01 – Broken Access Control****

- Are there authorization checks on any endpoint?
- Can an unauthenticated user call DELETE or POST endpoints?

****A03 – Injection****

- Are SQL queries built with string concatenation or ``fmt.Sprintf``?
- Are query parameters sanitized before use?

****A05 – Security Misconfiguration****

- Are error messages returned verbatim to HTTP clients?
- Does the server expose stack traces or internal paths?

****A07 – Identification and Authentication Failures****

- Is there any authentication middleware at all?

****A09 – Security Logging and Monitoring Failures****

- Are failed requests or suspicious inputs logged?

Output format

Write a report with this structure:

Security Audit Report

****Files reviewed****: list every file you read

For each finding:

****[SEVERITY] OWASP Category – Short title****

- File: ``path/to/file.go``, line N
- Description: what the vulnerability is
- Evidence: paste the relevant code snippet
- Recommendation: one concrete fix

Severity levels: CRITICAL, HIGH, MEDIUM, LOW, INFO

End with a ****Summary**** table: | Severity | Count |

Optional, before you test it: run `/mcp-coach 3` on your `description` field. It grades your trigger the way the A-or-B slide judged the two examples. It also predicts one moment where it would misfire.

Step 3 — Test by asking Claude directly (without explicitly triggering the subagent):

"Do a security audit of the bookstore API."

Watch the tool calls in the output. Claude will delegate to `security-auditor` automatically because the description matches. The audit runs in a separate context — your main conversation stays clean.

Done when: `bookstore-go/.claude/agents/security-auditor.md` exists, and you have a security-audit report produced by the subagent, not written by you.

Closing round (10 min: write 5 · popcorn 5)

Before the trainer popcorns the room, write down answers to these:

1. **MCP schema awareness:** did Claude call `get_table_definitions` before every query, or only once? What does that tell you about how Claude reuses tool results within a session?
2. **Subagent delegation:** did Claude delegate automatically, or did you have to trigger it explicitly? What would you change in the `description:` field to make auto-delegation more reliable?
3. **Real-world applications:** which MCP servers would save your team the most time, and what would you need to check before connecting one?
4. **Cost vs capability:** the security auditor uses Haiku to save cost. Name one task in your own workflow that a cheaper model could handle. Say why it would not lose quality.

Then start `/verify-exercise 5`. It checks your MCP registration and your subagent's file, trigger, and report. It grades by observable state, not by prose. Bring its report to the closing round if it finishes in time.

Done when: you have a written answer for all four, and `/verify-exercise 5` is running.

Trainer popcorns the room — no pairs, no prep time beyond what you wrote above. Close with **one take-away** you'd give someone who skipped this session.